TAREA GLOBAL: ESTRUCTURAS DE DATOS Y GESTIÓN DE ARCHIVOS EN PYTHON (UT3)

1. FICHA TÉCNICA DE LA ACTIVIDADES

Parámetro	Detalle
Módulo	Programación con Python (Optativa - ASIR)
Unidad	UT3. Estructuras de Datos, Cadenas y Gestión de Archivos
Duración	15 horas estimadas
Herramientas	Visual Studio Code, Python 3, Entorno GNU/Linux / Shell
Formato	Archivo comprimido .zip con scripts .py, ficheros de prueba y memoria técnica en PDF

2. DESCRIPCIÓN GENERAL Y OBJETIVOS

Esta unidad de trabajo profundiza en el almacenamiento y manipulación de datos en memoria y persistencia en disco. El alumnado aprenderá a seleccionar la estructura adecuada (listas, tuplas o diccionarios), procesar cadenas complejas y manipular ficheros del sistema de forma segura mediante control de excepciones (`try-except`) y gestores de contexto (`with open`), enfocándolo a tareas de administración de sistemas e infraestructura de red.

3. CUADERNO DE ACTIVIDADES

BLOQUE A: ESTRUCTURAS DE DATOS SECUENCIALES (LISTAS Y TUPLAS)

1. **Ejercicio 1: Análisis de Potencias (Cuadrados y Cubos)**
   Escribe un programa que inicialice una lista fija con 10 valores enteros (del 1 al 10). A continuación, recorre la lista e imprime en pantalla tres columnas formateadas: el número original, su cuadrado y su cubo.

Pista de Python: Puedes iterar con `for n in lista:` y utilizar *f-strings* con especificadores de alineación (ejemplo: `f"{n:<5} {n**2:<8} {n**3:<8}"`) para generar columnas parejas.

2. **Ejercicio 2: Gestión de Calificaciones Escolares**
   Diseña un programa que solicite al usuario las notas de 5 asignaturas de un alumno (valores entre 0.0 y 10.0). Al finalizar la recogida de datos, el programa debe mostrar por consola:
   • La lista completa de notas ingresadas.
   • La nota media del alumno.
   • La nota más alta (máxima) y la más baja (mínima).

   Pista de Python: Ve acumulando los valores con `lista.append(float(...))` y apóyate en las funciones nativas `sum()`, `len()`, `max()` y `min()`.

3. **Ejercicio 3: Inversión de Nombres de Equipos**
   Escribe un script que solicite al usuario el nombre de 5 servidores de red por teclado y los almacene en una lista. Copia estos elementos en una nueva lista pero en orden inverso (sin usar `.reverse()` ni modificadores de la original, hazlo mediante segmentación/slicing) y muestra la lista resultante por pantalla.

   Pista de Python: Recuerda que la sintaxis de paso negativo `servidores[::-1]` genera una nueva lista invertida sin mutar la fuente original.

4. **Ejercicio 4: Calendario de Sistemas**
   Crea un script que almacene los meses del año y sus respectivos días utilizando listas (supón que febrero siempre tiene 28 días). El programa pedirá al usuario un número de mes (del 1 al 12) y devolverá un mensaje indicando el nombre del mes y la cantidad de días que tiene. Si el número introducido está fuera de rango, debe informar del error.

   Pista de Python: Ten en cuenta el desfase de indexación en Python: el mes 1 corresponde al índice `0` (es decir, `meses[numero - 1]`).

5. **Ejercicio 5: Filtro Antimulticast / Purga de IPs Duplicadas**
   Crea un programa que simule la recogida de direcciones de red IP de origen en un switch. El programa pedirá de forma continua direcciones IP (strings) hasta que el usuario introduzca un número negativo (o la palabra "fin"). Posteriormente, el programa debe generar una nueva lista eliminando todas las IPs duplicadas y mostrarla ordenada de menor a mayor.

Pista de Python: Al recorrer la lista original, añade cada elemento a la nueva lista solo si se cumple `if ip not in lista_unicas:`. Después aplica `sorted()` o `lista_unicas.sort()`.

BLOQUE B: ESTRUCTURAS DE DATOS ASOCIATIVAS (DICCIONARIOS) Y CADENAS

6. **Ejercicio 6: Generador de Diccionario Cuadrático**
   Escribe un programa que solicite un número entero n por teclado. El script debe autogenerar un diccionario cuyas claves sean los números enteros desde el 1 hasta n, y sus valores correspondientes sean el cuadrado de dichas claves. Finalmente, muestra el diccionario por consola.

   Pista de Python: Puedes poblarlo dinámicamente con un bucle `for i in range(1, n + 1):` asignando `cuadrados[i] = i ** 2` o mediante un diccionario por comprensión.

7. **Ejercicio 7: Telemetría de Caracteres**
   Crea un programa que lea una cadena de texto (por ejemplo, una línea de log) introducida por teclado y devuelva un diccionario con la frecuencia de aparición de cada uno de los caracteres en dicha cadena.

   Pista de Python: Para cada carácter en la cadena puedes comprobar si ya existe en el diccionario o usar el método `conteo[char] = conteo.get(char, 0) + 1`.

8. **Ejercicio 8: Tarifas de Licencias Cloud**
   Declara un diccionario que almacene los nombres de diferentes suites de software cloud y sus precios de suscripción mensual por usuario (ej: `'SaaS_A': 15.5`, `'SaaS_B': 24.0`, etc.). El programa pedirá recursivamente el nombre del software y la cantidad de licencias necesarias. Mostrará el presupuesto total calculado. Si el software no se encuentra en el catálogo, debe lanzar un mensaje controlado y permitir otra consulta hasta que el usuario decida salir.

   Pista de Python: Valida la existencia de la clave antes de calcular con `if suite in catalogo:` o utilizando `catalogo.get(nombre)` verificando que no retorne `None`.

9. **Ejercicio 9: Notas con Estructuras Anidadas**
   Codifica un programa en Python que guarde el nombre del alumnado de un grupo de sistemas y sus respectivas notas de prácticas de administración. Un alumno puede tener un número variable de notas. Almacena la información en un diccionario cuyas claves sean los nombres de los alumnos y los valores sean listas con sus notas. El programa

solicitará el nombre y las notas (hasta que se introduzca un número negativo para terminar las notas de ese alumno). Al finalizar, el programa mostrará los nombres de los alumnos y la nota media obtenida por cada uno de ellos.

Pista de Python: Para recorrer e imprimir las medias recorre el diccionario con `for alumno, notas in registro.items():` y calcula la media con `sum(notas) / len(notas)` si la lista no está vacía.

10. **Ejercicio 10: Limpiador y Normalizador de Direcciones de Correo**
    En la administración de servidores de correo, a menudo se reciben listas crudas con espacios y mayúsculas desordenadas. Escribe un script que tome una cadena de correos electrónicos separados por punto y coma (`;`), elimine los espacios en blanco iniciales/finales de cada correo, los convierta por completo a minúsculas, y devuelva una lista limpia con las direcciones normalizadas.

    Pista de Python: Divide primero la cadena usando `cadena.split(';')` y aplica `correo.strip().lower()` a cada elemento resultante.

BLOQUE C: GESTIÓN DE ARCHIVOS Y EXCEPCIONES

11. **Ejercicio 11: Bitácora Diaria (El Diario)**
    Crea un programa para administradores que pida al usuario escribir un evento o incidencia por teclado y lo guarde en un archivo llamado `diario.txt` . Cada vez que se ejecute el programa, la nueva entrada debe anexarse al final del archivo con un salto de línea, sin sobrescribir las incidencias anteriores.

    Pista de Python: Abre el archivo con `with open("diario.txt", "a", encoding="utf-8") as f:` para usar el modo de adición (*append*) y añade explícitamente `\n` .

12. **Ejercicio 12: Lector Seguro de Cabecera y Cola**
    Crea de manera manual un archivo de texto llamado `configuracion.cfg` con al menos 5 líneas. Escribe un script en Python que intente abrir este archivo para lectura y muestre en pantalla únicamente la primera línea y la última línea del archivo. Controla posibles excepciones de acceso.

    Pista de Python: Usa `f.readlines()` para cargar todas las líneas en una lista y accede a `lineas[0]` y `lineas[-1]` , encerrando la lectura en un bloque `try - except FileNotFoundError:` .

13. **Ejercicio 13: Copia de Seguridad Resiliente**
    Desarrolla un programa de backup que lea el contenido de un archivo llamado `notas.txt` y realice una copia exacta en otro archivo llamado `notas_backup.txt`. Debes prever y capturar con `try-except` la excepción en caso de que el archivo de origen no exista en el directorio de trabajo, informando amigablemente al usuario del error en lugar de colgar el programa.

    Pista de Python: Estructura el flujo con `try:` abriendo origen en modo `"r"` y destino en modo `"w"`, capturando específicamente `except FileNotFoundError:`.

14. **Ejercicio 14: Contador de Palabras de Logs**
    Crea un programa que lea un archivo de logs de texto plano (`logs.txt`) y calcule el número total de palabras contenidas en él. Utiliza el método `.split()` de cadenas de texto para segmentar las palabras de forma óptima.

    Pista de Python: Puedes leer todo el fichero en un solo string con `f.read()` y luego invocar `len(contenido.split())`, ya que `split()` sin argumentos agrupa y limpia cualquier separación de espacios o saltos de línea.

15. **Ejercicio 15: Rotación e Historial de Ejecuciones (Logs)**
    Crea un programa de telemetría de scripts que, cada vez que sea invocado por el sistema de tareas programadas (cron), escriba en un archivo `ejecuciones.log` la frase "Ejecución exitosa" junto con un número de secuencia progresivo (ej: "Ejecución 1", "Ejecución 2", etc.), respetando los registros históricos anteriores utilizando el modo append (`a`).

    Pista de Python: Antes de escribir en modo `"a"`, puedes verificar si el archivo existe; si existe, léelo previamente en modo `"r"` con `len(f.readlines())` para conocer el número correlativo que corresponde asignar.

16. **Ejercicio 16: Exportación de Auditoría a JSON (Ampliación)**
    Crea un script que almacene un diccionario con el estado de 3 servicios del servidor (ej: `{"nginx": "active", "ssh": "active", "mysql": "failed"}`) y lo guarde en un archivo estructurado `auditoria.json` usando el módulo nativo `json` de Python.

    Pista de Python: Importa `json` y utiliza la función `json.dump(datos, f, indent=4)` dentro de un bloque `with open("auditoria.json", "w")`.

2.1.2. EJERCICIOS DE CONSOLIDACIÓN (NIVEL INTERMEDIO)

A. **Ejercicio A: Gestión de Inventario de Hardware (Diccionarios y Archivos)**
Objetivo: Crear un programa para controlar el stock de componentes de un almacén.
Técnica: Leer un archivo `stock.txt` (formato `componente:cantidad`) y cargarlo en un diccionario. El programa debe permitir añadir nuevos componentes o actualizar cantidades y guardar los cambios.
Teoría aplicada: Diccionarios (mutabilidad), `with open` (lectura/escritura), `split()` de cadenas y bucles interactivos.

Pista de Python: Para procesar cada línea usa `linea.strip().split(":")`, guarda el valor numérico con `int(cantidad)` y al finalizar escribe recorriendo `diccionario.items()` con `f.write(f"{k}:{v}\n")`.

B. **Ejercicio B: Clasificador de Niveles de Log (Procesamiento de Strings)**
Objetivo: Analizar `logs_sistema.txt` y clasificar líneas según su severidad (`INFO`, `WARNING`, `ERROR`).
Técnica: Leer el archivo línea a línea, usar métodos de cadenas para detectar la palabra clave, guardar mensajes en tres listas y mostrar el conteo.
Teoría aplicada: Métodos de cadenas (`find`, `upper`/`lower`, o el operador `in`), listas, bucles y control de flujo.

Pista de Python: Normaliza la línea con `linea.upper()` antes de comprobar pertenencia para evitar fallos si los logs mezclan mayúsculas y minúsculas.

C. **Ejercicio C: Registro de Usuarios y Permisos (Tuplas y Filtrado)**
Objetivo: Filtrar una lista de usuarios de un sistema.
Técnica: Usar una lista de tuplas con el formato `(nombre_usuario, nivel_acceso)`. El script pide un nivel de acceso y muestra solo los usuarios que cumplen el filtro, guardando el resultado en `filtrado.txt`.
Teoría aplicada: Tuplas (inmutabilidad), listas (recorrido), condicionales, gestores de contexto y escritura en archivos.

Pista de Python: Puedes desempaquetar las tuplas directamente en el bucle: `for usuario, nivel in usuarios:` y comprobar si `nivel.lower() == filtro.lower()` antes de escribir en disco.

(Nota: Estas actividades se corregirán gradualmente en las sesiones prácticas del aula para verificar la correcta asimilación e implementación de las estructuras de datos, persistencia en ficheros y manejo seguro de excepciones).